STAR Gateway API Reference

Go Package Documentation

February 2026

Contents

1 Package: cmd.virtual_rx72n

Virtual RX72N - Hardware-in-the-Loop Simulator

This program simulates the RX72N motor controller firmware for testing and development without physical hardware. It listens on a Unix domain socket and responds to Gateway commands.

Usage:

```
go run ./cmd/virtual_rx72n/
```

The simulator will:
1. Create a Unix socket at /tmp/star_rx72n.sock
2. Listen for connections from the Gateway
3. Echo back modified data to prove it's the simulator
4. Handle Ctrl+C gracefully

STAR Project - Texas A&M University January 2026

1.0.1 CONSTANTS

1.0.2 const (

SocketPath = "/tmp/star_rx72n.sock"

MaxFrameSize = frame.MaxFrameSize

SimulatorMarker = 0xFF

SignalBufferSize = 1

PingPayloadSize = 4

DefaultSimLatencyMs = 10

MinSimLatencyMs = 0

MaxSimLatencyMs = 1000

SeqMask = 0xFFFF

EnvSimLatencyMs = "SIM_LATENCY_MS")

2 Package: internal.app

Package app encapsulates the main application logic for the star-gateway.

STAR Project - Texas A&M University January 2026

2.0.1 FUNCTIONS

2.0.2 func Run(ctx context.Context, config Config) error

```
Run starts the STAR Gateway application with the given configuration.

This is the main entry point that orchestrates the complete application
lifecycle:
 1. Logger initialization
 2. Transport layer setup (SPI/USB with automatic failover)
 3. Message dispatcher initialization
 4. Service layer initialization (gRPC and HTTP/WebSocket)
 5. Server startup (background goroutines)
 6. Signal handling (graceful shutdown on SIGINT/SIGTERM)

The function blocks until a shutdown signal is received or a fatal error
occurs.
```

2.0.3 TYPES

2.0.4 type Config struct {

SimulationMode bool SocketPath string TransportMode manager.TransportMode }

```
Config holds the application configuration.
```

2.0.5 type Servers struct {

HTTPServer *http.Server GRPCServer *grpc.Server }

3 Package: internal.controller

3.0.1 FUNCTIONS

3.0.2 func ArcadeDrive(linearVel, angularVel float32) (float32, float32)

```
ArcadeDrive converts gamepad stick inputs to left/right motor velocities.
linearVel: [-1.0, 1.0] (forward/backward) angularVel: [-1.0, 1.0]
(right/left) Returns: (leftMotor, rightMotor) in range [-1.0, 1.0]
```

3.0.3 TYPES

3.0.4 type Handler struct {

}

3.0.5 func NewHandler() *Handler

3.0.6 func NewHandlerWithGateway(gatewaySvc *service.GatewayService) *Handler

3.0.7 func (h *Handler) GetLastState() *starv1.ControllerState

3.0.8 func (h *Handler) GetSafeState() *starv1.ControllerState

```
GetSafeState returns the last state, or a zero state if the last message was
received more than 200ms ago.
```

3.0.9 func (h *Handler) ServeHTTP(w http.ResponseWriter, r *http.Request)

4 Package: internal.dispatcher

Package dispatcher implements a centralized message router for the star-gateway.

The Dispatcher solves the concurrency race condition where multiple services (MotorControlService, TelemetryService) all call HARQ.Receive() in parallel, causing frame stealing and deserialization errors.

Architecture:
1. Dispatcher owns the single HARQ.Receive() loop
2. Services subscribe to specific message types
3. Dispatcher demultiplexes messages via WireMessage.oneof
4. Subscribed channels receive only their type

STAR Project - Texas A&M University January 2026

4.0.1 CONSTANTS

4.0.2 const (

DefaultShutdownTimeout = 5 * time.Second

DefaultSubscriberBufferSize = 100

DefaultReceiveInterval = 10 * time.Millisecond)

```
Configuration constants.
```

4.0.3 VARIABLES

4.0.4 var (

ErrDispatcherStopped = errors.New("dispatcher: dispatcher is stopped")

ErrHARQNil = errors.New("dispatcher: HARQ handler is nil")

ErrLoggerNil = errors.New("dispatcher: logger is nil"))

```
Predefined errors for dispatcher operations.
```

4.0.5 TYPES

4.0.6 type Config struct {

ShutdownTimeout time.Duration

ReceiveInterval time.Duration }

```
Config holds dispatcher configuration parameters.
```

4.0.7 type DispatchedMessage struct {

WireMsg *starv1.WireMessage Metadata *harq.FrameMetadata }

```
DispatchedMessage bundles a WireMessage with frame metadata for diagnostics.
Exported so services can access the metadata.
```

4.0.8 type Dispatcher interface {

Subscribe(msgType MessageType) <-chan *DispatchedMessage

Unsubscribe(msgType MessageType, ch <-chan *DispatchedMessage)

Start(ctx context.Context) error

Stop() error

GetState() State }

```
Dispatcher defines the interface for centralized message routing.
```

4.0.9 func NewDispatcher(h harq.HARQ, logger *slog.Logger, cfg *Config) (Dispatcher, error)

```
NewDispatcher creates a new message dispatcher. The harq handler and logger
are required.
```

4.0.10 type MessageType uint32

```
MessageType identifies a message type in the WireMessage union.
```

4.0.11 const (

MessageTypeUnknown MessageType = iota

MessageTypeVelocityCommand

MessageTypeEmergencyStopCommand

MessageTypeMotorPowerCommand

MessageTypeTelemetryData

MessageTypeEncoderData

MessageTypePidConfig)

4.0.12 type State uint8

```
State represents the dispatcher operational state.
```

4.0.13 const (

StateIdle State = iota

StateRunning

StateStopping

StateStopped)

5 Package: internal.fec

Chase Combiner implementation for HARQ Type I.

Chase Combining stores soft bits from failed transmission attempts and combines them element-wise (addition) before decoding. This improves the effective SNR with each retransmission.

Example:

```
Transmission 1: soft bits S1 (decode fails)
Transmission 2: soft bits S2 (decode fails)
Combined: S_combined[i] = S1[i] + S2[i] (clamped to int16 range)
Decode: Viterbi(S_combined) -> success
```

STAR Project - Texas A&M University December 2025

Convolutional encoder implementation for rate-1/2, K=7 code.

Generator polynomials (NASA standard):
- G1 = 171 octal = 0b1111001 = 121 decimal
- G2 = 133 octal = 0b1011011 = 91 decimal

The encoder uses a 6-bit shift register (K-1 = 6 memory elements). For each input bit, two output bits are generated.

STAR Project - Texas A&M University December 2025

Package fec implements Forward Error Correction for the HARQ protocol.

This package provides a rate-1/2 convolutional encoder (K=7) with soft Viterbi decoding, designed for Chase Combining HARQ where soft bits from multiple transmissions are accumulated before decoding.

FEC Parameters:
- Rate: 1/2 (2 output bits per input bit)
- Constraint Length: K=7 (64 states)
- Generator Polynomials: G1=171o, G2=133o (NASA standard)

Reference: docs/sections/01_nanopb_protocol.tex (Layer 3: HARQ)

STAR Project - Texas A&M University December 2025

Viterbi decoder implementation for rate-1/2, K=7 convolutional code.

This decoder supports both soft and hard decision decoding. Soft decision decoding provides approximately 2dB coding gain over hard decision.

The decoder uses the add-compare-select (ACS) algorithm with traceback to find the maximum likelihood path through the trellis.

STAR Project - Texas A&M University December 2025

5.0.1 CONSTANTS

5.0.2 const (

ConstraintLength = 7

NumStates = 1 << (ConstraintLength - 1)

G1Octal = 0171

G2Octal = 0133

TailBits = ConstraintLength - 1)

```
    Convolutional encoder constants.
```

5.0.3 const (

TracebackDepth = 5 * ConstraintLength

MaxPathMetric = 0x7FFFFFFF)

5.0.4 const DefaultMaxCombines = 3

```
    DefaultMaxCombines is the default maximum number of combining attempts.
```

5.0.5 VARIABLES

5.0.6 var (

ErrCombinerFull = errors.New("fec: combiner full, max attempts reached")

ErrCombinerEmpty = errors.New("fec: combiner empty")

ErrSoftBitsLengthMismatch = errors.New("fec: soft bits length mismatch"))

```
    Combiner error codes.
```

5.0.7 var (

ErrInvalidInput = errors.New("fec: invalid input data")

ErrInvalidSoftBits = errors.New("fec: invalid soft bits")

ErrDecodeFailed = errors.New("fec: decode failed")

ErrLengthMismatch = errors.New("fec: length mismatch"))

```
    FEC error codes.
```

5.0.8 TYPES

5.0.9 type ChaseCombiner struct {

}

```
    ChaseCombiner implements soft bit combining for HARQ Type I.
```

5.0.10 func NewChaseCombiner(config *CombinerConfig) *ChaseCombiner

```
    NewChaseCombiner creates a new Chase Combiner with the given config.
```

5.0.11 func (c *ChaseCombiner) Add(softBits []SoftBit) error

```
    Add accumulates soft bits from a new transmission attempt. The first call
    sets the expected length for subsequent calls.
```

5.0.12 func (c *ChaseCombiner) CanAdd() bool

```
    CanAdd returns true if more transmissions can be combined.
```

5.0.13 func (c *ChaseCombiner) Combined() []SoftBit

```
    Combined returns the accumulated soft bits ready for decoding. The
    accumulated values are scaled and clamped back to SoftBit range.
```

5.0.14 func (c *ChaseCombiner) Count() int

```
    Count returns the number of transmissions combined so far.
```

5.0.15 func (c *ChaseCombiner) MaxCombines() int

```
    MaxCombines returns the maximum number of combining attempts.
```

5.0.16 func (c *ChaseCombiner) Reset()

```
Reset clears the combining buffer for a new frame.
```

5.0.17 type CombinerConfig struct {

MaxCombines int }

```
CombinerConfig holds configuration for the Chase Combiner.
```

5.0.18 type ConvolutionalEncoder struct {

}

```
ConvolutionalEncoder implements a rate-1/2, K=7 convolutional encoder.
```

5.0.19 func NewConvolutionalEncoder() *ConvolutionalEncoder

```
NewConvolutionalEncoder creates a new convolutional encoder.
```

5.0.20 func (e *ConvolutionalEncoder) Encode(data []byte) ([]byte, error)

```
Encode applies convolutional encoding to the input data. For each input bit,
two output bits are generated. Tail bits are appended to flush the encoder
to the zero state.

Output length = (inputLen * 8 + TailBits) * 2 bits, packed into bytes.
```

5.0.21 func (e *ConvolutionalEncoder) OutputLength(inputLen int) int

```
OutputLength returns the encoded output length in bytes for a given input
length.
```

5.0.22 func (e *ConvolutionalEncoder) Rate() float64

```
Rate returns the code rate (0.5 for rate-1/2).
```

5.0.23 func (e *ConvolutionalEncoder) Reset()

```
Reset resets the encoder state to zero.
```

5.0.24 type Decoder interface {

DecodeSoft(softBits []SoftBit, expectedOutputLen int) ([]byte, int, error)

DecodeHard(data []byte, expectedOutputLen int) ([]byte, error)

InputLength(outputLen int) int }

```
Decoder defines the interface for FEC decoding.
```

5.0.25 type Encoder interface {

Encode(data []byte) ([]byte, error)

Rate() float64

OutputLength(inputLen int) int }

```
Encoder defines the interface for FEC encoding.
```

5.0.26 type SoftBit int8

```
SoftBit represents a soft decision value for a received bit. Range: -127 to
+127
  - Negative values indicate a '0' bit (more negative = higher confidence)
  - Positive values indicate a '1' bit (more positive = higher confidence)
  - Zero indicates an erasure (no information)
```

```
The magnitude represents confidence level (log-likelihood ratio
approximation).
```

5.0.27 const (

SoftBitMax SoftBit = 127

SoftBitMin SoftBit = −127

SoftBitErasure SoftBit = 0)

5.0.28 func HardToSoft(bit uint8) SoftBit

```
HardToSoft converts a hard bit (0 or 1) to maximum confidence soft bit.
```

5.0.29 func (s SoftBit) Confidence() uint8

```
Confidence returns the absolute confidence level (0-127).
```

5.0.30 func (s SoftBit) ToBit() uint8

```
ToBit converts a SoftBit to a hard decision (0 or 1).
```

5.0.31 type SoftCombiner interface {

Add(softBits []SoftBit) error

Combined() []SoftBit

Count() int

Reset() }

```
SoftCombiner accumulates soft bits from multiple transmission attempts.
Used for Chase Combining in HARQ Type I.
```

5.0.32 type ViterbiDecoder struct {

}

```
ViterbiDecoder implements soft Viterbi decoding for rate-1/2, K=7 code.
```

5.0.33 func NewViterbiDecoder() *ViterbiDecoder

```
NewViterbiDecoder creates a new Viterbi decoder.
```

5.0.34 func (d *ViterbiDecoder) DecodeHard(data []byte, expectedOutputLen int) ([]byte, error)

```
DecodeHard decodes hard decision bits. Converts hard bits to maximum
confidence soft bits and decodes. expectedOutputLen specifies the expected
decoded data length in bytes (0 = auto-detect).
```

5.0.35 func (d *ViterbiDecoder) DecodeSoft(softBits []SoftBit, expectedOutputLen int) ([]byte, int, error)

```
DecodeSoft decodes soft bits using the Viterbi algorithm. expectedOutputLen
specifies the expected decoded data length in bytes. If 0, it's
auto-calculated from the soft bits length. Returns the decoded data,
final path metric, and any error.
```

5.0.36 func (d *ViterbiDecoder) InputLength(outputLen int) int

```
InputLength returns the expected soft bits length for a given output length.
```

6 Package: internal.frame

Package frame provides CRC-32 implementation.

Package frame provides frame decoding for the wire protocol.

STAR Project - Texas A&M University December 2025

Package frame defines the wire protocol frame structure and decoding logic.

The protocol uses a custom framing format: [SYNC(2)][SEQ(2)][LEN(2)][TYPE(1)][FLAGS(1)] [PAYLOAD(0-1KB)][CRC-32(4)]

Key features: - 2-byte sync word for frame alignment (0x55AA) - 16-bit sequence number - CRC-32 (IEEE 802.3) data integrity - Max payload 1024 bytes - Type field for message classification

STAR Project - Texas A&M University

Package frame provides frame encoding for the wire protocol.

STAR Project - Texas A&M University December 2025

Package frame defines the wire protocol frame structure for RPi5 <-> RX72N communication.

Frame format:

```
    [SYNC (2B, LE)][SEQ (2B, LE)][LEN (2B, LE)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32
(4B)]
```

All multi-byte header fields (SYNC, SEQ, LEN) are little-endian by project convention. The CRC-32 uses the IEEE 802.3 polynomial and bit-ordering; the resulting 32-bit CRC value is serialized in little-endian to match the header field convention.

STAR Project - Texas A&M University January 2026

Package frame provides a streaming decoder for the STAR protocol.

6.0.1 CONSTANTS

6.0.2 const (

SyncWord uint16 = 0x55AA

SyncSize = 2

SeqSize = 2 LenSize = 2 TypeSize = 1 FlagsSize = 1

HeaderSize = SeqSize + LenSize + TypeSize + FlagsSize

CRCSize = 4

MaxPayloadSize = 1024

MaxFrameSize = SyncSize + HeaderSize + MaxPayloadSize + CRCSize

MinFrameSize = SyncSize + HeaderSize + CRCSize

EmptyPayloadLength = 0)

```
    Frame structure constants.
```

6.0.3 VARIABLES

6.0.4 var (

ErrNotImplemented = errors.New(“not implemented”) ErrInvalidSync = errors.New(“invalid sync word”) ErrInvalidCRC = errors.New(“CRC validation failed”) ErrPayloadTooLarge = errors.New(“payload exceeds maximum size”) ErrFrameTooShort = errors.New(“frame data too short”))

```
Predefined errors.
```

6.0.5 var EmptyPayload = []byte{}

```
EmptyPayload is a pre-allocated empty byte slice for frames with no payload
(ACK/NACK/PING). Using this constant avoids repeated allocations of empty
slices.
```

6.0.6 var ErrInvalidLength = errors.New(“frame length exceeds maximum”)

```
ErrInvalidLength indicates frame length field exceeds limits.
```

6.0.7 var ErrSyncLoss = errors.New(“frame sync lost after max search”)

```
ErrSyncLoss indicates frame synchronization was lost.
```

6.0.8 TYPES

6.0.9 type CRCError struct {

Sequence uint16 }

```
CRCError is returned by the stream decoder when CRC validation fails.
It carries the parsed sequence number so callers can send a NACK.
```

6.0.10 func (e *CRCError) Error() string

6.0.11 func (e *CRCError) Unwrap() error

```
Unwrap returns the underlying sentinel error (ErrInvalidCRC) to support Go
1.13+ error unwrapping with errors.Is and errors.As.
```

6.0.12 type DecodeMetrics struct {

TotalFrames uint64 SyncErrors uint64 CRCErrors uint64 LengthErrors uint64 LastSuccess time.Time }

```
DecodeMetrics tracks decoder health statistics.
```

6.0.13 type Decoder interface {

Decode(data []byte) (*Frame, error) }

```
Decoder defines the interface for frame decoding.
```

6.0.14 type DefaultDecoder struct{}

```
DefaultDecoder implements the Decoder interface.
```

6.0.15 func NewDecoder() *DefaultDecoder

```
NewDecoder creates a new DefaultDecoder.
```

6.0.16 func (d *DefaultDecoder) Decode(data []byte) (*Frame, error)

```
Decode parses wire format bytes into a Frame.

Wire format (all fields in little-endian per IEEE 802.3):

    [SYNC (2B)][SEQ (2B)][LEN (2B)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32 (4B)]
```

```
CRC-32 is validated over SYNC + Header + Payload.
```

6.0.17 type DefaultEncoder struct{}

```
DefaultEncoder implements the Encoder interface.
```

6.0.18 func NewEncoder() *DefaultEncoder

```
NewEncoder creates a new DefaultEncoder.
```

6.0.19 func (e *DefaultEncoder) Encode(frame *Frame) ([]byte, error)

```
Encode serializes a Frame into wire format bytes.

Wire format:

    [SYNC (2B)][SEQ (2B)][LEN (2B)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32 (4B)]

Header fields (SYNC, SEQ, LEN, TYPE, FLAGS) are encoded in little-endian by
project choice for consistency across platforms.

CRC-32 uses the IEEE 802.3 polynomial (0xEDB88320) with bit/byte ordering as
defined by the standard. The CRC is calculated over SYNC + Header + Payload.
```

6.0.20 type Encoder interface {

Encode(frame *Frame) ([]byte, error) }

```
Encoder defines the interface for frame encoding.
```

6.0.21 type Flags uint8

```
Flags contains frame-level control flags.
```

6.0.22 const (

FlagNone Flags = 0x00 FlagRequiresAck Flags = 0x01 FlagRetransmit Flags = 0x02 FlagPriority Flags = 0x04 FlagFECEnabled Flags = 0x08 FlagSoftNACK Flags = 0x10)

6.0.23 type Frame struct {

Header Header

Type Type

Payload []byte

CRC uint32

Retransmits int

FECDecoded bool }

```
Frame represents a complete communication frame.
```

6.0.24 func NewFrame(frameType Type, payload []byte) (*Frame, error)

```
NewFrame creates a new Frame with the given type and payload.
```

6.0.25 type Header struct {

Sequence uint16

Length uint16

Flags Flags }

```
Header contains the frame header fields.
```

6.0.26 type StreamDecoder struct {

}

```
StreamDecoder reads and decodes frames from a continuous stream. It handles
synchronization loss, variable length payloads, and CRC validation.

Thread Safety: StreamDecoder enforces single-reader semantics. Only one
goroutine should call Decode() at a time. Concurrent calls will serialize
via internal mutex. GetMetrics() may briefly block if Decode() is active,
but this is typically negligible (<1ms) since metrics updates are fast.

The mutex protects both buffer state and metrics to ensure consistency.
This design trades some concurrency for simplicity and correctness.
```

6.0.27 func NewStreamDecoder(r io.Reader) *StreamDecoder

```
NewStreamDecoder creates a new decoder reading from r.
```

6.0.28 func (d *StreamDecoder) Decode() (*Frame, error)

```
Decode blocks until a valid frame is read or an error occurs. It
automatically handles resynchronization on stream errors.
```

6.0.29 func (d *StreamDecoder) GetMetrics() DecodeMetrics

```
GetMetrics returns the current health metrics.
```

6.0.30 type Type uint8

```
Type indicates the type of frame being transmitted. Not serialized in Header
struct, but part of the wire header.
```

6.0.31 const (

FrameTypePing Type = 0x00 FrameTypePong Type = 0x01 FrameTypeResetAck Type = 0xFE FrameTypeReset Type = 0xFF

FrameTypeCommand Type = 0x10 FrameTypeResponse Type = 0x11 FrameTypeAck Type = 0x12 FrameTypeNack Type = 0x13

FrameTypeUnknown Type = 0x14)

6.0.32 func (ft Type) String() string

7 Package: internal.harq

Package harq implements Hybrid Automatic Repeat reQuest (HARQ) protocol for reliable frame delivery over the SPI transport.

The HARQ mechanism uses Chase Combining (Type I) with the following properties:

- 16-bit sequence numbers with wraparound
- ACK/NACK response frames
- Configurable retry count and timeout
- Duplicate frame detection
- Forward Error Correction (FEC) with soft Viterbi decoding
- Soft bit combining across retransmissions

Chase Combining stores soft bits from failed transmissions and combines them element-wise before re-attempting Viterbi decoding. This provides approximately 2dB coding gain per additional transmission.

Reference: docs/sections/01_nanopb_protocol.tex (Layer 3: HARQ)

STAR Project - Texas A&M University December 2025

7.0.1 CONSTANTS

7.0.2 const (

DefaultMaxRetries = 3

DefaultTimeout = 10 * time.Millisecond

DefaultSequenceMax = 0xFFFF)

```
HARQ protocol constants from specification.
```

7.0.3 const (

PriorityValueEmergency uint8 = 0 PriorityValueHigh uint8 = 1 PriorityValueNormal uint8 = 2)

```
Protocol wire format values for priority levels
```

7.0.4 VARIABLES

7.0.5 var (

ErrNotImplemented = errors.New(“harq: not implemented”)

ErrMaxRetriesExceeded = errors.New(“harq: max retries exceeded”)

ErrTimeout = errors.New(“harq: timeout waiting for acknowledgment”)

ErrInvalidSequence = errors.New(“harq: invalid sequence number”)

ErrDuplicateFrame = errors.New(“harq: duplicate frame detected”)

ErrTransportNil = errors.New(“harq: transport is nil”)

ErrEncoderNil = errors.New(“harq: encoder is nil”)

ErrDecoderNil = errors.New(“harq: decoder is nil”)

ErrFECEncoderNil = errors.New(“harq: FEC encoder is nil”)

ErrFECDecoderNil = errors.New(“harq: FEC decoder is nil”)

ErrUnexpectedFrameType = errors.New(“harq: unexpected frame type”)

ErrInErrorState = errors.New(“harq: in error state, call Reset() first”)

ErrDecodeFailed = errors.New(“harq: FEC decode failed after combining”)

ErrNilContext = errors.New(“harq: context is nil”))

```
Predefined errors for HARQ operations.
```

7.0.6 TYPES

7.0.7 type ChaseCombining struct {

}

```
ChaseCombining implements the HARQ interface using Chase Combining (Type I).
```

7.0.8 func NewChaseCombining(

config *Config, t transport.Device, encoder frame.Encoder, decoder frame.Decoder, fecEncoder fec.Encoder, fecDecoder fec.Decoder,) *ChaseCombining

```
NewChaseCombining creates a new Chase Combining HARQ handler. If config is
nil, uses DefaultConfig(). The transport, frame encoder/decoder, and FEC
encoder/decoder are required dependencies.
```

7.0.9 func (h *ChaseCombining) Config() *Config

```
Config returns the current HARQ configuration.
```

7.0.10 func (h *ChaseCombining) GetExpectedLenForTesting() int

```
GetExpectedLenForTesting returns the expected decoded length for testing.
```

7.0.11 func (h *ChaseCombining) GetRxSequence() uint16

```
GetRxSequence returns the expected RX sequence number.
```

7.0.12 func (h *ChaseCombining) GetState() State

```
GetState returns the current HARQ state.
```

7.0.13 func (h *ChaseCombining) GetTxSequence() uint16

```
GetTxSequence returns the current TX sequence number.
```

7.0.14 func (h *ChaseCombining) Receive(ctx context.Context) (*ReceiveResult, error)

```
Receive waits for data with HARQ handling. On receive, attempts FEC decode.
On failure, stores soft bits and waits for retransmission. Combines soft
bits from multiple transmissions. Validates CRC and sequence number,
sends ACK for valid frames.
```

7.0.15 func (h *ChaseCombining) Reset()

```
Reset resets the HARQ state machine to initial state.
```

7.0.16 func (h *ChaseCombining) Send(ctx context.Context, data []byte, p ...Priority) error

```
Send transmits data with HARQ reliability. Applies FEC encoding, wraps in
a command frame, and waits for ACK. Retransmits the same encoded frame on
timeout or NACK up to MaxRetries. Priority defaults to PriorityNormal if not
specified.
```

7.0.17 func (h *ChaseCombining) SetExpectedLenForTesting(expectedLen int)

```
SetExpectedLenForTesting sets the expected decoded length for testing.
```

7.0.18 func (h *ChaseCombining) SetStateForTesting(state State, txSeq, rxSeq uint16, retryCount int)

```
SetStateForTesting sets internal state for testing purposes. This method
should only be used in tests to simulate various states.
```

7.0.19 type Config struct {

MaxRetries int

Timeout time.Duration

FECEnabled bool }

```
Config holds HARQ configuration parameters.
```

7.0.20 func DefaultConfig() *Config

```
DefaultConfig returns the default HARQ configuration.
```

7.0.21 type FrameMetadata struct {

Sequence uint16 Type frame.Type ReceivedAt time.Time

Retransmits int FECDecoded bool

PathMetric int CombiningCount int }

```
FrameMetadata contains frame-level information for diagnostics.
```

7.0.22 type HARQ interface {

Send(ctx context.Context, data []byte, p ...Priority) error

Receive(ctx context.Context) (*ReceiveResult, error)

GetState() State

GetTxSequence() uint16

GetRxSequence() uint16

Reset() }

```
HARQ defines the interface for the HARQ protocol handler.
```

7.0.23 type Priority uint8

```
Priority represents the priority level of a HARQ transmission.
```

7.0.24 const (

PriorityEmergency Priority = 0 PriorityHigh Priority = 1 PriorityNormal Priority = 2)

7.0.25 type ReceiveResult struct {

Payload []byte Metadata FrameMetadata }

```
ReceiveResult bundles payload with frame metadata.
```

7.0.26 type State uint8

```
State represents the HARQ state machine state.
```

7.0.27 const (

StateIdle State = iota

StateWaitingAck

StateRetransmitting

StateCombining

StateError)

7.0.28 func (s State) String() string

```
String returns the string representation of a State.
```

8 Package: internal.link

Package link provides link layer implementations for the STAR Gateway.

CDCLink implements a lightweight link layer for USB CDC transport. Unlike SPI (which uses full HARQ with retransmissions and FEC), USB CDC relies on hardware-level reliability. CDCLink provides:

- CRC32 validation (end-to-end integrity)
- Sequence tracking via shared SessionState (prevents "Handoff Problem")
- Gap tolerance (accepts sequence gaps <10 frames for packet loss recovery)
- Send serialization via sendMutex (prevents out-of-order transmission)
- NO retransmission (failures propagate to TransportManager for failover)
- NO FEC encoding/decoding (USB handles error correction)
- NO ACK/NACK frames (USB provides implicit ACK)

Critical Fixes Implemented:

- Fix #1: Shared SessionState prevents sequence resets during transport

```
switching
```

- Fix #5: sendMutex ensures atomic sequence+write (no out-of-order frames)
- Fix #6: Gap tolerance allows recovery from rare USB packet loss
- Fix #7: Context timeout prevents blocked writes on USB disconnect

STAR Project - Texas A&M University January 2026

Package link provides link layer implementations for the STAR Gateway.

SPILink implements a full HARQ (Hybrid ARQ) link layer for SPI transport. Unlike CDCLink (lightweight USB), SPILink uses:

- ACK/NACK handshake for reliability
- Retransmission (up to 3 attempts)
- Optional FEC (Forward Error Correction) with Viterbi decoder
- Optional Chase Combining for soft-bit accumulation
- Sequence tracking via shared SessionState (prevents "Handoff Problem")

Critical Fixes Implemented:

- Fix #1: Shared SessionState prevents sequence resets during transport

```
switching
```

- Fix #2: Smart drain logic via FailureType (ACK timeout vs hard IO error)
- Fix #5: sendMutex ensures atomic sequence+write (no out-of-order frames)

STAR Project - Texas A&M University February 2026

8.0.1 VARIABLES

8.0.2 var (

ErrSPITransportNotInitialized = errors.New("SPI transport not initialized") ErrMaxRetriesExceeded = errors.New("maximum retries exceeded") ErrACKTimeout = errors.New("ACK timeout"))

```
Predefined errors for SPI link operations.
```

8.0.3 var (

ErrCDCTransportNotInitialized = errors.New("CDC transport not initialized"))

```
Predefined errors.
```

8.0.4 TYPES

8.0.5 type CDCLink struct {
}

```
CDCLink implements a lightweight link layer for USB CDC transport.
Unlike SPILink (full HARQ), CDCLink relies on USB hardware reliability.
Sequences are managed by shared SessionState (prevents Handoff Problem).
```

8.0.6 func NewCDCLink(t transport.Device, session *manager.SessionState) (*CDCLink, error)

```
NewCDCLink creates a new CDC link layer with shared session state. CRITICAL:
session parameter MUST be shared across all transports (USB and SPI) to
prevent sequence desynchronization during transport switching.
```

8.0.7 func (c *CDCLink) GetRxSequence() uint16

```
GetRxSequence implements harq.HARQ interface. Delegates to shared
SessionState.
```

8.0.8 func (c *CDCLink) GetState() harq.State

```
GetState implements harq.HARQ interface. CDCLink has no retransmission state
machine, always returns Idle or Error.
```

8.0.9 func (c *CDCLink) GetTxSequence() uint16

```
GetTxSequence implements harq.HARQ interface. Delegates to shared
SessionState.
```

8.0.10 func (c *CDCLink) Receive(ctx context.Context) (*harq.ReceiveResult, error)

```
Receive implements harq.HARQ interface. Validates CRC32 (decoder does this)
and sequence using shared SessionState. CRITICAL FIX #6: Gap tolerance
accepts sequence gaps <10 frames.
```

8.0.11 func (c *CDCLink) Reset()

```
Reset implements harq.HARQ interface. Delegates to shared SessionState
(resets both TX and RX sequences to 0). CRITICAL: This is called after
receiving RESET_ACK from RX72N to synchronize sequences after Gateway
restart (prevents restart deadlock - FIX #4).
```

8.0.12 func (c *CDCLink) Send(ctx context.Context, data []byte, p ...harq.Priority) error

```
Send implements harq.HARQ interface. Uses shared SessionState for sequence
management (CRITICAL FIX #1). Serializes Send operations with sendMutex
(CRITICAL FIX #5).
```

8.0.13 func (c *CDCLink) SendWithSeq(ctx context.Context, data []byte, seq uint16, flags frame.Flags) error

```
SendWithSeq sends data with the specified sequence number and flags.
CRITICAL FIX #5: sendMutex ensures atomic sequence+write (no out-of-order
frames). CRITICAL FIX #7: Context timeout prevents blocked writes on USB
disconnect.
```

8.0.14 func (c *CDCLink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

```
SendWithType sends data with a specific frame type (PING, PONG, RESET,
etc.). This allows explicit control over the frame type for protocol
messages. Phase 3: Frame Type API implementation.
```

8.0.15 type SPILink struct {

}

```
SPILink implements a full HARQ link layer for SPI transport.

Pattern: Based on CDCLink structure but with HARQ additions:
  - ACK/NACK handshake
  - Retransmission logic
  - FEC encoding/decoding (optional)
  - Chase Combining for soft-bit accumulation (optional)
```

8.0.16 func NewSPILink(t transport.Device, session *manager.SessionState, config *SPILinkConfig) (*SPILink, error)

```
NewSPILink creates a new SPI link layer with shared session state.

CRITICAL: session parameter MUST be shared across all transports (USB and
SPI) to prevent sequence desynchronization during transport switching.

Parameters:
  - t: SPI transport device
  - session: Shared SessionState instance (from
    TransportManager.GetSessionState())
  - config: Configuration (nil = use defaults)

Returns:
  - *SPILink: Initialized SPI link
  - error: Initialization error (nil/invalid parameters)
```

8.0.17 func (s *SPILink) GetRxSequence() uint16

```
GetRxSequence implements harq.HARQ interface. Delegates to shared
SessionState.
```

8.0.18 func (s *SPILink) GetState() harq.State

```
GetState implements harq.HARQ interface. Returns current HARQ state machine
state.
```

8.0.19 func (s *SPILink) GetTxSequence() uint16

```
GetTxSequence implements harq.HARQ interface. Delegates to shared
SessionState.
```

8.0.20 func (s *SPILink) Receive(ctx context.Context) (*harq.ReceiveResult, error)

```
Receive implements harq.HARQ interface with ACK/NACK response generation.

Protocol Flow:
 1. Decode frame via StreamDecoder (validates CRC32 automatically)
 2. CRITICAL FIX: Check if ACK/NACK frame - dispatch to Send() if so
 3. Validate sequence using shared SessionState
 4. CRITICAL FIX: On duplicate - send ACK (not NACK) to stop retransmit loop
 5. Optional: FEC decode payload (if FlagFECEnabled)
 6. On success: Send ACK if required, return payload
 7. CRITICAL FIX: Don't fail if ACK send fails (just log)
 8. On failure: Send NACK, return error

Thread Safety: receiveMutex prevents concurrent calls that would race on
s.decoder. The decoder maintains internal buffer state and is NOT safe for
concurrent access.
```

8.0.21 func (s *SPILink) Reset()

```
Reset implements harq.HARQ interface. Delegates to shared SessionState
(resets both TX and RX sequences to 0). Also resets FEC combiner if enabled.
```

8.0.22 func (s *SPILink) Send(ctx context.Context, data []byte, p ...harq.Priority) error

```
Send implements harq.HARQ interface with ACK/NACK handshake and
retransmission. Uses frame.FrameTypeCommand for all data frames.

Thread Safety: sendMutex ensures atomicity (sequence assignment + Send)
```

8.0.23 func (s *SPILink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

```
SendWithType sends data with explicit frame type (PING, PONG, RESET, etc.).
This allows explicit control over the frame type for protocol messages.
Phase 3: Frame Type API implementation.

Thread Safety: sendMutex ensures atomicity (sequence assignment + Send)
```

8.0.24 type SPILinkConfig struct {

EnableFEC bool

MaxRetries int

ACKTimeout time.Duration }

```
SPILinkConfig holds configuration parameters for SPILink.
```

8.0.25 func DefaultSPILinkConfig() *SPILinkConfig

```
DefaultSPILinkConfig returns the default configuration for SPILink.
```

9 Package: internal.manager

Package manager ...

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

9.0.1 CONSTANTS

9.0.2 const (

SPIProbeTimeout = 50 * time.Millisecond

SPIProbePayloadSize = 4

SPIProbeTestMarker = 0xDEADBEEF)

9.0.3 const (

DefaultPingInterval = 1 * time.Second

DefaultFailureTimeout = 200 * time.Millisecond)

9.0.4 const (

DefaultConsecutiveMissThreshold = 1)

```
    WDT Timing Coordination

    The RX72N hardware Watchdog Timer (WDT) has an approximate timeout of 1
    second. Our software heartbeat MUST detect failure significantly faster to
    prevent unintended hardware reboots.

    Timing budget (dual detection strategy):

        WDT timeout:       ~1000ms (hardware, configured in RX72N firmware)
        Implicit timeout:    200ms (any frame resets timer via OnFrameReceived)
        Check interval:       50ms (failureTimeout / 4, for timely detection)
        PING interval:      1000ms (1 Hz, idle-link probe -- rare when data is flowing)
        Miss threshold:    1 PING  (single unanswered PING triggers failover)
        Worst-case detect:  ~250ms (implicit timeout + one check interval)
        Safety margin:        ~4x  (250ms << 1000ms WDT)

    How it works:

        Active link:  Data frames (including ACK/NACK) keep lastSeen fresh.
                      PING fires every pingInterval since last valid PONG (independent timer).
                      Implicit timeout never fires because lastSeen stays current.
        Zombie link:  Stale OS-buffered frames keep lastSeen fresh, but firmware is dead.
                      PING fires at pingInterval since last PONG; no PONG arrives;
```

```
                counter-based failover triggers. Implicit timeout cannot catch this
                because lastSeen is continuously refreshed by the buffered frames.
    Dead link:  No frames of any kind; implicit timeout fires at 200ms (faster than
                PING at 1s, so implicit path is the fast-path for hard failures).

WARNING: Do NOT increase DefaultFailureTimeout above 500ms without first
verifying the RX72N WDT timeout. The implicit timeout is the critical path.

To verify RX72N WDT timeout:
 1. Check star-rx72n-firmware WDT configuration registers (WDTCR)
 2. WDT clock source and divider determine actual timeout
 3. Typical: PCLKB/8192 with 16-bit counter ~ 1.09s at 60MHz PCLKB
```

9.0.5 const (

DefaultTargetDevice = “ttyACM0”)

9.0.6 const (

TransportNameUSB = “usb”

TransportNameSPI = “spi”)

9.0.7 const (

PriorityUSB = 10

PrioritySPI = 5)

9.0.8 const (

InvalidTransportModeError = “invalid transport mode: %q”)

9.0.9 const MaxGapTolerance uint16 = 10

```
MaxGapTolerance is the maximum number of skipped frames allowed before
rejecting a received sequence as a large gap (transport failure). It
controls how many frames may be lost (for example on lightweight USB) while
still accepting and catching up the RX sequence.
```

9.0.10 const SessionIDPayloadSize = 4

```
SessionIDPayloadSize is the byte size of the session ID in RESET/RESET_ACK
payloads. The session ID is encoded as a 4-byte little-endian uint32.
```

9.0.11 TYPES

9.0.12 type Config struct {

Mode TransportMode

HealthCheckInterval time.Duration

FailureThreshold int

SwitchTimeout time.Duration

EnableHotPlug bool

HotPlugPollInterval time.Duration

USBVID uint16

USBPID uint16

FailbackDamping time.Duration

HealthLatencyThreshold time.Duration

HealthLossThreshold float64 }

```
Config holds the configuration for the TransportManager.
```

9.0.13 func DefaultConfig() *Config

```
DefaultConfig returns a Config with production-ready default settings.

Default behavior:
  - Auto mode: prefer USB, fallback to SPI
  - Health checks every 5 seconds
  - Failover after 3 consecutive failures
  - 500ms switch timeout
  - Hot-plug detection enabled

Note: USBPID should be updated with the actual RX72N Product ID once known.
```

9.0.14 func (c *Config) Validate() error

```
Validate checks if the configuration is valid.
```

9.0.15 type ConfigError struct {

Field string Reason string }

```
ConfigError represents a configuration validation error.
```

9.0.16 func (e *ConfigError) Error() string

9.0.17 type FailureType int

```
FailureType indicates why a transport switch was triggered. This determines
whether the TransportManager should drain in-flight operations before
switching (CRITICAL FIX #2: Smart Drain Logic).
```

9.0.18 const (

FailureTypeGraceful FailureType = iota

FailureTypeTimeout

FailureTypeIOError

FailureTypeHealthCheck

FailureTypeHotPlugRemove)

9.0.19 func (ft FailureType) String() string

```
String returns a human-readable failure type name.
```

9.0.20 type HARQReader struct {

}

```
HARQReader adapts a harq.HARQ interface to io.Reader. It buffers payloads
received from HARQ and provides them as a byte stream.

Architecture Note: This adapter exists because harq.HARQ.Receive() returns
complete HARQ packets (which may contain multiple frames or partial frames),
while StreamDecoder expects a continuous byte stream. The adapter bridges
this impedance mismatch by:
 1. Calling HARQ.Receive() to get packet payloads (blocking up to
    readTimeout)
```

```
    2. Buffering the payload bytes
    3. Exposing them as an io.Reader for StreamDecoder to parse frame
       boundaries

    Thread Safety: Read() is safe for concurrent calls but only one will make
    progress at a time due to internal mutex. This matches io.Reader contract.
```

9.0.21 func NewHARQReader(h harq.HARQ) *HARQReader

```
    NewHARQReader creates a new adapter.
```

9.0.22 func (r *HARQReader) Read(p []byte) (int, error)

```
    Read implements io.Reader.
```

9.0.23 type HealthMetrics struct {

LastSuccess time.Time

LastFailure time.Time

LastRecovery time.Time

ConsecutiveFailures int

TotalSent uint64

TotalReceived uint64

TotalErrors uint64

AvgLatency time.Duration

PacketLossRate float64

IsHealthy bool

}

```
    HealthMetrics tracks the operational health of a transport.
```

9.0.24 func NewHealthMetrics() *HealthMetrics

```
    NewHealthMetrics creates a new HealthMetrics instance with default values.
```

9.0.25 type HealthMonitor struct {

}

```
    HealthMonitor periodically checks the health of inactive transports.

    It runs in a background goroutine and probes transports that are not
    currently active. This allows the TransportManager to detect when a
    previously failed transport has recovered.
```

9.0.26 func NewHealthMonitor(interval time.Duration) *HealthMonitor

```
    NewHealthMonitor creates a new HealthMonitor with the given check interval.
```

9.0.27 func (hm *HealthMonitor) Run(ctx context.Context, tm *TransportManager)

```
    Run starts the health monitoring loop. It exits when ctx is cancelled.
```

9.0.28 type HeartbeatManager struct {

}

```
    HeartbeatManager implements hybrid implicit/explicit connectivity detection.
```

```
Hybrid Detection Strategy:
  - Implicit: Updates LastSeen on ANY valid frame (telemetry, commands,
    ACKs)
  - Explicit: Sends PING when idle >1s (rare idle-link probe)
  - Failure: Declares link dead if no frames >200ms (implicit timeout fires
    first)

Benefits:
  - Minimal bandwidth overhead (only PING when idle)
  - Fast failure detection (200ms vs 5s polling)
  - No false positives from buffered OS data (counter prevents replay)

Thread Safety: All methods use mutex protection for concurrent access.
```

9.0.29 func NewHeartbeatManager(pingInterval, failureTimeout time.Duration, onFailure func()) (*HeartbeatManager, error)

```
NewHeartbeatManager creates a new heartbeat manager with the specified
timeouts.

Parameters:
  - pingInterval: How long the link must be idle before sending explicit
    PING (recommended 1s)
  - failureTimeout: How long without any frames before declaring failure
    (recommended 200ms)
  - onFailure: Callback function to invoke when link is declared dead

Returns an error if:
  - pingInterval or failureTimeout is <= 0
  - onFailure is nil (required for triggering failover)

Note: pingInterval may be greater than failureTimeout. In the dual-detection
model, the implicit timeout (failureTimeout) is the primary detection
mechanism -- any valid frame resets the timer. PING is a rare idle-link
probe that fires only when the link has been silent for pingInterval.
```

9.0.30 func (hm *HeartbeatManager) OnFrameReceived()

```
OnFrameReceived updates the LastSeen timestamp for implicit heartbeat
detection.

This method should be called by TransportManager.Receive() for valid
non-PONG frames, including:
  - PING frames (from remote side)
  - Command frames
  - Response frames

PONG frames should use [OnPongReceived] for explicit counter validation.

By updating on any frame, we avoid sending unnecessary PINGs when the link
is actively transferring data. Also clears the failureTriggered flag to
allow future failover if the link fails again.

Thread Safety: Uses mutex to protect lastSeen timestamp and failureTriggered
flag.
```

9.0.31 func (hm *HeartbeatManager) OnPongReceived(payload []byte) bool

```
OnPongReceived validates a PONG payload against the last sent PING counter.
```

```
This performs both implicit and explicit heartbeat detection:
  - Implicit: Always updates lastSeen and clears failureTriggered (link is
    alive)
  - Explicit: Validates the 4-byte counter matches lastPingSent

On counter match:
  - Resets consecutiveMisses to 0
  - Updates lastValidPong timestamp
  - Clears pendingPing flag

On counter mismatch (stale/replayed PONG):
  - Logs a warning with both counters
  - Does NOT reset consecutiveMisses (link may be degraded)
  - lastSeen is still updated (link is physically alive)

Returns true if the counter matched, false otherwise.

Thread Safety: Uses mutex to protect all state fields.
```

9.0.32 func (hm *HeartbeatManager) Run(ctx context.Context, tm *TransportManager)

```
Run starts the heartbeat monitoring loop as a background goroutine.

This method:
 1. Periodically checks elapsed time since lastSeen (every failureTimeout/4)
 2. Triggers failover if idle > failureTimeout (200ms)
 3. Sends PING if idle > pingInterval (1s) and link not yet failed

The check interval is derived from failureTimeout to ensure timely
detection. With 200ms timeout and check every 50ms, worst-case detection is
~250ms.

The loop runs until ctx is cancelled. It should be started as a goroutine:

    go heartbeat.Run(ctx, transportManager)

Thread Safety: Safe to call concurrently with OnFrameReceived().
```

9.0.33 type HotPlugDetector struct {

}

```
HotPlugDetector monitors for USB device add/remove events.

On Linux, this uses inotify via fsnotify to watch /sys/bus/usb/devices for
device creation/removal. This provides instant detection (<50ms) compared to
polling-based approaches.

Phase 3: Full inotify implementation for instant hot-plug failover.
```

9.0.34 func NewHotPlugDetector(pollInterval time.Duration, vid, pid uint16) *HotPlugDetector

```
NewHotPlugDetector creates a new HotPlugDetector. The pollInterval parameter
is ignored in the inotify implementation but kept for API compatibility.
```

9.0.35 func (hpd *HotPlugDetector) Run(ctx context.Context, eventHandler func(HotPlugEvent))

```
Run starts the hot-plug detection loop using inotify. It watches
/sys/bus/usb/devices for Create/Remove events and calls eventHandler when
```

```
    the target device is added or removed.

    On non-Linux platforms or if fsnotify fails, it falls back to a no-op
    (graceful degradation).
```

9.0.36 type HotPlugEvent struct {

Action string

Device string

Timestamp time.Time

VendorID uint16

ProductID uint16 }

```
    HotPlugEvent represents a USB device add/remove event.
```

9.0.37 type SessionState struct {

}

```
    SessionState holds sequence numbers shared across all transports. When
    Gateway fails over from USB (seq=105) to SPI, the SPI link MUST continue
    from seq=106, not reset to 0. SessionState ensures all transports share the
    same sequence counter.
```

9.0.38 func NewSessionState() *SessionState

```
    NewSessionState creates a new SessionState with initial sequence 0.
```

9.0.39 func (s *SessionState) ActivateBarrier(sessionID uint32)

```
    ActivateBarrier enables the reset barrier for the given session ID.
    While the barrier is active, ValidateRxSequence rejects all frames (they are
    stale), and only a RESET_ACK with a matching session ID should be accepted.

    Must be called BEFORE sending the RESET frame to ensure no stale frames from
    the USB FIFO buffer are processed during the handshake.

    Thread Safety: Uses mutex to ensure atomicity.
```

9.0.40 func (s *SessionState) DeactivateBarrier()

```
    DeactivateBarrier disables the reset barrier. Called after the reset
    handshake completes (success or failure) to resume normal frame processing.
    Should be called in a defer to guarantee cleanup.

    Thread Safety: Uses mutex to ensure atomicity.
```

9.0.41 func (s *SessionState) GetRxSequence() uint16

```
    GetRxSequence returns the current RX sequence (for diagnostics).

    Thread Safety: Uses mutex for consistent reads.
```

9.0.42 func (s *SessionState) GetSessionID() uint32

```
    GetSessionID returns the current session ID (for diagnostics).

    Thread Safety: Uses mutex for consistent reads.
```

9.0.43 func (s *SessionState) GetTxSequence() uint16

```
GetTxSequence returns the current TX sequence (for diagnostics).

Thread Safety: Uses mutex for consistent reads.
```

9.0.44 func (s *SessionState) IncrementSessionID() uint32

```
IncrementSessionID atomically increments and returns the new session ID.
Called at the start of each reset handshake to generate a unique identifier
for matching RESET frames with their corresponding RESET_ACK responses.

Thread Safety: Uses mutex to ensure atomicity.
```

9.0.45 func (s *SessionState) IsBarrierActive() bool

```
IsBarrierActive returns whether the reset barrier is currently active.

Thread Safety: Uses mutex for consistent reads.
```

9.0.46 func (s *SessionState) NextTxSequence() uint16

```
NextTxSequence atomically increments and returns the next TX sequence.

This method is called by all transport link layers (CDCLink, SPILink) to
generate the next sequence number for outgoing frames. The shared counter
ensures continuity across transport switches.

Thread Safety: Uses mutex to ensure atomicity across concurrent goroutines.
```

9.0.47 func (s *SessionState) Reset()

```
Reset resets both sequences to 0 (used when RX72N resets).

CRITICAL FIX #4: This is called after receiving a RESET_ACK from RX72N to
synchronize sequences after Gateway restart.

Thread Safety: Uses mutex to ensure atomicity.
```

9.0.48 func (s *SessionState) ValidateResetAck(payload []byte) bool

```
ValidateResetAck checks if a RESET_ACK payload matches the active barrier's
session ID. Returns true only if:
  - The barrier is currently active
  - The payload is exactly SessionIDPayloadSize (4) bytes
  - The little-endian decoded session ID matches the barrier's expected
    session ID

Thread Safety: Uses mutex to ensure atomicity.
```

9.0.49 func (s *SessionState) ValidateRxSequence(seq uint16) bool

```
ValidateRxSequence checks if the received sequence is valid. Returns true if
valid (exact match or small gap), false if duplicate or large gap.

CRITICAL FIX #6: Allow small gaps for lightweight USB protocol (no
retransmission). If USB drops a single frame due to rare bit flip, we accept
the gap and continue rather than permanently stalling the link.

Gap Tolerance:
  - diff == 0: Exact match (most common case) -> Accept
  - 0 < diff < 10: Small gap (packet loss) -> Accept and catch up
  - diff >= 10: Large gap (transport failure) -> Reject
  - diff is large positive (near 65535): Likely duplicate from wraparound ->
```

```
    Reject

Thread Safety: Uses mutex to ensure atomicity across concurrent goroutines.
```

9.0.50 type State uint8

```
State represents the internal state of the TransportManager.
```

9.0.51 const (

StateInitializing State = iota

StateActiveUSB

StateActiveSPI

StateSwitchingToUSB

StateSwitchingToSPI

StateDegraded

StateFailed)

9.0.52 func (s State) String() string

```
String returns a human-readable state name.
```

9.0.53 type TransportManager struct {

}

```
TransportManager manages multiple transports with automatic failover and
health monitoring.

It implements the harq.HARQ interface and acts as a transparent proxy to the
active transport. The Dispatcher interacts with TransportManager exactly
as it would with a single HARQ instance, while TransportManager handles all
transport selection, health monitoring, and failover logic.

Key features:
  - Priority-based transport selection (USB=PriorityUSB, SPI=PrioritySPI)
  - Health monitoring with configurable failure thresholds
  - Automatic failover when active transport fails
  - USB hot-plug detection (add/remove events)
  - Zero-downtime transport switching with pause-drain-resume
  - Thread-safe operations with fine-grained locking

Thread Safety:
  - All public methods are thread-safe
  - Uses RWMutex for high read concurrency
  - Operations lock prevents race conditions during switching
```

9.0.54 func NewTransportManager(config *Config) *TransportManager

```
NewTransportManager creates a new TransportManager with the given
configuration.

If config is nil, DefaultConfig() is used. The manager is created in
StateInitializing and must be started with Start().

Example:

    tm := manager.NewTransportManager(manager.DefaultConfig())
```

```
    tm.RegisterTransport("spi", spiLink, manager.PrioritySPI)
    tm.RegisterTransport("usb", usbLink, manager.PriorityUSB)
    tm.Start(ctx)
```

9.0.55 func (tm *TransportManager) ForceSwitch(transportName string) error

```
ForceSwitch manually switches to a specific transport, bypassing priority
logic.

This method is useful for testing or manual override scenarios.

Returns an error if:
  - The specified transport is not registered
  - The specified transport is not available
  - The switch operation fails
```

9.0.56 func (tm *TransportManager) GetActiveTransport() string

```
GetActiveTransport returns the name of the currently active transport.
Returns empty string if no transport is active.
```

9.0.57 func (tm *TransportManager) GetAvailableTransports() []string

```
GetAvailableTransports returns a list of currently available transport
names. Only includes transports marked as Available.
```

9.0.58 func (tm *TransportManager) GetHealthMetrics() map[string]*HealthMetrics

```
GetHealthMetrics returns a copy of health metrics for all registered
transports.
```

9.0.59 func (tm *TransportManager) GetInternalState() State

```
GetInternalState returns the current internal State (for debugging/testing).
```

9.0.60 func (tm *TransportManager) GetRxSequence() uint16

```
GetRxSequence returns the RX sequence number from the active transport.
Returns 0 if no transport is active.
```

9.0.61 func (tm *TransportManager) GetSessionState() *SessionState

```
GetSessionState returns the shared SessionState instance. CRITICAL FIX #1:
All transports (USB CDC and SPI) MUST use the same SessionState to prevent
sequence desynchronization during transport switching (Handoff Problem).

Usage in gateway.go:

    tm := manager.NewTransportManager(config)
    session := tm.GetSessionState()
    cdcLink, _ := link.NewCDCLink(cdcTransport, session)
    spiLink, _ := link.NewSPILink(spiTransport, session)
    tm.RegisterTransport("usb", cdcLink, manager.PriorityUSB)
    tm.RegisterTransport("spi", spiLink, manager.PrioritySPI)
```

9.0.62 func (tm *TransportManager) GetState() harq.State

```
GetState returns the current harq.State mapped from the internal State.

Mapping:
  - StateActiveUSB, StateActiveSPI -> harq.StateIdle
  - StateSwitching* -> harq.StateRetransmitting
  - StateFailed -> harq.StateError
```

9.0.63 func (tm *TransportManager) GetTxSequence() uint16

```
GetTxSequence returns the TX sequence number from the active transport.
Returns 0 if no transport is active.
```

9.0.64 func (tm *TransportManager) Receive(ctx context.Context) (*harq.ReceiveResult, error)

```
Receive proxies the Receive operation to the active transport with health
tracking.

This method implements the harq.HARQ interface. It blocks if a transport
switch is in progress, then resumes once switching completes.

Control frames (PING, PONG, ACK, NACK, RESET_ACK) are consumed internally:
  - PING: auto-replies with PONG (echoing payload), updates implicit
    heartbeat
  - PONG: validates counter against last PING (explicit heartbeat)
  - ACK/NACK/RESET_ACK: logged and consumed

Only data frames (COMMAND, RESPONSE) are returned to the caller.
```

9.0.65 func (tm *TransportManager) RegisterTransport(name string, transport harq.HARQ, priority int)

```
RegisterTransport adds a transport to the manager with the given priority.

Higher priority transports are preferred (e.g., USB=10, SPI=5).
If this transport has higher priority than the current active transport,
an automatic switch will be attempted in the background.

This method is thread-safe and can be called while the manager is running.

Parameters:
  - name: Transport identifier (e.g., "spi", "usb")
  - transport: HARQ implementation
  - priority: Selection priority (higher = preferred)
```

9.0.66 func (tm *TransportManager) Reset()

```
Reset resets the active transport to a clean state. This is a pass-through
to the active transport's Reset method.
```

9.0.67 func (tm *TransportManager) Send(ctx context.Context, data []byte, p ...harq.Priority) error

```
Send proxies the Send operation to the active transport with health
tracking.

This method implements the harq.HARQ interface. It blocks if a transport
switch is in progress, then resumes once switching completes.

Returns an error if:
  - No active transport is available (StateFailed)
  - The active transport's Send operation fails

Failed operations increment the failure counter. When the failure threshold
is reached, an automatic failover is triggered in the background.
```

9.0.68 func (tm *TransportManager) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

```
SendWithType sends data with an explicit frame type (PING, PONG, RESET,
etc.). This is a Phase 3 API for protocol messages that require specific
frame types.

If the active transport supports SendWithType(), it uses that method.
Otherwise, it falls back to Send() (CDCLink and SPILink both support
SendWithType).

This method blocks if a transport switch is in progress, then resumes once
switching completes.
```

9.0.69 func (tm *TransportManager) Start(ctx context.Context) error

```
Start initializes and starts the TransportManager background routines.

This method:
 1. Starts the health monitor for inactive transports
 2. Starts the hot-plug detector (if enabled)
 3. Selects the initial active transport

The provided context is used for graceful shutdown. Call Stop() to clean up.

Returns an error if:
  - No transports are available and mode is ModeForceUSB or ModeForceSPI
  - Config validation fails
```

9.0.70 func (tm *TransportManager) Stop() error

```
Stop gracefully shuts down the TransportManager.

This method:
 1. Cancels all background routines
 2. Waits for all goroutines to exit
 3. Resets all registered transports

It is safe to call Stop multiple times.
```

9.0.71 type TransportMode string

```
TransportMode defines the transport selection strategy.
```

9.0.72 const (

ModeAuto TransportMode = “auto”

ModeForceUSB TransportMode = “force-usb”

ModeForceSPI TransportMode = “force-spi”)

9.0.73 func ParseTransportMode(s string) (TransportMode, error)

```
ParseTransportMode converts a string to a TransportMode. Returns ModeAuto
and an error if the input is invalid.
```

9.0.74 func (tm TransportMode) IsValid() bool

```
IsValid checks if the TransportMode is one of the defined constants.
```

9.0.75 type TransportWrapper struct {

Name string

Transport harq.HARQ

Decoder *frame.StreamDecoder

Health *HealthMetrics

Priority int

Available bool }

```
    TransportWrapper wraps a transport with health metrics and metadata.
```

10 Package: internal.server

Package server provides HTTP and gRPC server construction and lifecycle management.

This package follows a two-phase construction pattern that separates server configuration from server startup:

1. Constructor phase: NewHTTPServer() / NewGRPCServer() create configured

   ```
   servers without side effects (no network I/O, no goroutines).
   ```

2. Run phase: RunHTTPServer() / RunGRPCServer() start servers in background

   ```
   goroutines with automatic graceful shutdown via context cancellation.
   ```

Two-Phase Construction Benefits

This pattern enables:

- Testable handler/service wiring without opening network ports
- Inspecting server configuration before startup
- Separating config errors (constructor) from runtime errors (Run)
- Graceful shutdown via context cancellation (no manual shutdown calls)

HTTP Server Example

```
// Phase 1: Construct server with handler
mux := http.NewServeMux()
mux.HandleFunc("/health", healthHandler)

const defaultHTTPPort = ":8080"

config := &server.HTTPConfig{
    ListenAddr:   defaultHTTPPort,
    ReadTimeout:  DefaultReadTimeout,
    WriteTimeout: DefaultWriteTimeout,
    Handler:      mux,
}

httpSrv, err := server.NewHTTPServer(config, logger)
if err != nil {
    return fmt.Errorf("failed to create HTTP server: %w", err)
}

// Phase 2: Start server (non-blocking)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

errChan, err := server.RunHTTPServer(ctx, httpSrv, logger)
if err != nil {
    return fmt.Errorf("failed to start HTTP server: %w", err)
}

// Monitor for runtime errors
go func() {
    if err := <-errChan; err != nil {
        logger.Error("HTTP server error", slog.String("error", err.Error()))
    }
}()

// Graceful shutdown: cancel context
```

```
    // (RunHTTPServer automatically shuts down when ctx is cancelled)
    cancel()
```

gRPC Server Example

```
    // Phase 1: Construct server with service registration
    const bytesPerKiB = 1024
    const bytesPerMiB = bytesPerKiB * 1024
    const defaultMaxMessageMultiplier = 10
    const defaultMaxMessageSize = defaultMaxMessageMultiplier * bytesPerMiB

    config := &server.GRPCConfig{
        ListenAddr:     DefaultGRPCListenAddr,
        MaxMessageSize: defaultMaxMessageSize,
        ServiceRegistrar: func(srv *grpc.Server) error {
            starv1.RegisterMotorControlServiceServer(srv, motorService)
            starv1.RegisterTelemetryServiceServer(srv, telemetryService)
            return nil
        },
    }

    grpcSrv, err := server.NewGRPCServer(config, logger)
    if err != nil {
        return fmt.Errorf("failed to create gRPC server: %w", err)
    }

    // Phase 2: Start server (non-blocking)
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    errChan, err := server.RunGRPCServer(ctx, grpcSrv, logger)
    if err != nil {
        return fmt.Errorf("failed to start gRPC server: %w", err)
    }

    // Monitor for runtime errors
    go func() {
        if err := <-errChan; err != nil {
            logger.Error("gRPC server error", slog.String("error", err.Error()))
        }
    }()

    // Graceful shutdown: cancel context
    // (RunGRPCServer automatically shuts down with 5s timeout fallback)
    cancel()
```

Testing Without Network I/O

The two-phase pattern enables testing handler wiring without opening ports:

```
    // Test HTTP handler
    config := &server.HTTPConfig{
        ListenAddr: ":8080",
        Handler:    myHandler,
    }
    httpSrv, _ := server.NewHTTPServer(config, testLogger)

    req := httptest.NewRequest("GET", "/test", nil)
    recorder := httptest.NewRecorder()
```

```
    httpSrv.Handler.ServeHTTP(recorder, req)

    // Assert on recorder.Code, recorder.Body
```

Graceful Shutdown

Both RunHTTPServer and RunGRPCServer automatically handle graceful shutdown:

- HTTP: Calls Server.Shutdown() with DefaultShutdownTimeout
- gRPC: Calls GracefulStop() with DefaultGracefulStopTimeout, falls back to `Stop()`
- Shutdown is triggered automatically when the context passed to Run is `cancelled`
- No manual shutdown calls required

STAR Project - Texas A&M University February 2026

10.0.1 CONSTANTS

10.0.2 const (

DefaultListenAddr = ":8080" DefaultReadTimeout = 10 * time.Second DefaultWriteTimeout = 10 * time.Second DefaultGRPCListenAddr = ":50051" DefaultGRPCMaxMessageSize = 10 * 1024 * 1024)

10.0.3 const (

DefaultShutdownTimeout = 5 * time.Second)

10.0.4 const DefaultGracefulStopTimeout = 5 * time.Second

```
    DefaultGracefulStopTimeout is the maximum time allowed for gRPC server
    graceful shutdown.
```

10.0.5 FUNCTIONS

10.0.6 func NewGRPCServer(config *GRPCConfig, logger *slog.Logger) (*grpc.Server, error)

```
    NewGRPCServer creates a configured grpc.Server and registers services.

    The server is configured with:
      - MaxRecvMsgSize and MaxSendMsgSize from config.MaxMessageSize
      - Services registered via config.ServiceRegistrar callback

    Returns error if:
      - Config validation fails
      - ServiceRegistrar callback returns error

    Example:

        config := &GRPCConfig{
            ListenAddr:     ":50051",
            MaxMessageSize: 10 * 1024 * 1024,
            ServiceRegistrar: func(srv *grpc.Server) error {
                starv1.RegisterMotorControlServiceServer(srv, motorService)
                return nil
            },
```

```
    }
    grpcSrv, err := NewGRPCServer(config, logger)
```

10.0.7 func NewHTTPServer(config *HTTPConfig, logger *slog.Logger) (*http.Server, error)

```
NewHTTPServer creates a configured http.Server without starting it. This
enables testing handler wiring without opening network ports.

The server is configured with:
  - Address from config.ListenAddr
  - Handler from config.Handler (typically http.ServeMux)
  - Timeouts from config (ReadTimeout, WriteTimeout)
  - Error logger that uses slog

Returns error if config validation fails.
```

10.0.8 func RunGRPCServer(ctx context.Context, srv *grpc.Server, addr string, logger *slog.Logger) (<-chan error, error)

```
RunGRPCServer starts the gRPC server in a background goroutine and returns
immediately.

The function:
 1. Creates a TCP listener on addr
 2. Starts server in background goroutine via srv.Serve()
 3. Spawns a shutdown watcher goroutine that triggers graceful shutdown on
    ctx.Done()

Returns:
  - Error channel for async error reporting (buffered, size 1)
  - Immediate error if listener creation fails

Graceful shutdown:
  - Triggered automatically when ctx is cancelled
  - Calls srv.GracefulStop() with grpcShutdownTimeout (5 seconds)
  - Waits for in-flight RPCs to complete
  - Falls back to srv.Stop() on timeout (forcefully terminates active RPCs)

The error channel receives:
  - nil on clean shutdown
  - error if srv.Serve() fails

Example:

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    errChan, err := RunGRPCServer(ctx, grpcSrv, ":50051", logger)
    if err != nil {
        return fmt.Errorf("failed to start: %w", err)
    }

    // Monitor for runtime errors
    go func() {
        if err := <-errChan; err != nil {
            logger.Error("server error", slog.String("error", err.Error()))
        }
    }()
```

```
    // Trigger graceful shutdown
    cancel()
```

10.0.9 func RunHTTPServer(ctx context.Context, srv *http.Server, logger *slog.Logger) (<-chan error, error)

```
RunHTTPServer starts the HTTP server in a background goroutine and returns
immediately.

The function:
 1. Creates a TCP listener on srv.Addr
 2. Starts server in background goroutine via srv.Serve()
 3. Spawns a shutdown watcher goroutine that triggers graceful shutdown on
    ctx.Done()

Returns:
  - Error channel for async error reporting (buffered, size 1)
  - Immediate error if listener creation fails

Graceful shutdown:
  - Triggered automatically when ctx is cancelled
  - Calls srv.Shutdown() with httpShutdownTimeout (5 seconds)
  - Allows in-flight requests to complete
  - After timeout, forcefully closes connections

The error channel receives:
  - nil on clean shutdown
  - error if srv.Serve() fails (excluding http.ErrServerClosed)

Example:

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    errChan, err := RunHTTPServer(ctx, httpSrv, logger)
    if err != nil {
        return fmt.Errorf("failed to start: %w", err)
    }

    // Monitor for runtime errors
    go func() {
        if err := <-errChan; err != nil {
            logger.Error("server error", slog.String("error", err.Error()))
        }
    }()

    // Trigger graceful shutdown
    cancel()
```

10.0.10 TYPES

10.0.11 type GRPCConfig struct {

ListenAddr string

MaxMessageSize int

ServiceRegistrar func(*grpc.Server) error }

```
GRPCConfig holds the configuration for gRPC server construction.
```

10.0.12 func DefaultGRPCConfig() *GRPCConfig

```
DefaultGRPCConfig returns a GRPCConfig with production-ready defaults.
ServiceRegistrar must be set by the caller as it's application-specific.
```

10.0.13 func (c *GRPCConfig) Validate() error

```
Validate checks if the GRPCConfig is valid and returns an error if
validation fails.
```

10.0.14 type HTTPConfig struct {

ListenAddr string

ReadTimeout time.Duration

WriteTimeout time.Duration

Handler http.Handler }

```
HTTPConfig holds the configuration for HTTP server construction.
```

10.0.15 func DefaultHTTPConfig() *HTTPConfig

```
DefaultHTTPConfig returns an HTTPConfig with production-ready defaults.
Handler must be set by the caller as it's application-specific.
```

10.0.16 func (c *HTTPConfig) Validate() error

```
Validate checks if the HTTPConfig is valid and returns an error if
validation fails. This matches the existing config validation pattern (see
manager.Config.Validate).
```

11 Package: internal.service

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University December 2025

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

11.0.1 CONSTANTS

11.0.2 const (

MaxMotors = 4

MinMotorID = 0

MaxMotorID = 3)

11.0.3 const (

MinRateHz = 1 MaxRateHz = 100 DefaultRateHz = 10)

11.0.4 TYPES

11.0.5 type ConfigurationService struct {

starv1.UnimplementedConfigurationServiceServer

}

```
ConfigurationService implements the ConfigurationServiceServer interface.
This service handles runtime configuration management including NVS
persistence.

Architecture: - Uses WireMessage wrapper for protocol multiplexing - Uses
Dispatcher for centralized message routing - Validates all configuration
parameters before applying - TODO: Integrate HARQ for actual firmware
communication
```

11.0.6 func NewConfigurationService(h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *ConfigurationService

```
NewConfigurationService creates a new ConfigurationService. Requires
HARQ handler for sending requests, Dispatcher for receiving responses,
and logger.
```

11.0.7 func (s *ConfigurationService) GetConfiguration(ctx context.Context, req *starv1.GetConfigurationRequest) (*starv1.GetConfigurationResponse, error)

```
GetConfiguration retrieves the current system configuration from the RX72N
firmware. TODO: Implement proper request via HARQ once firmware integration
is complete. For now, returns mock configuration until firmware integration
is ready.
```

11.0.8 func (s *ConfigurationService) GetMotorPidConfig(ctx context.Context, req *starv1.GetMotorPidConfigRequest) (*starv1.GetMotorPidConfigResponse, error)

```
GetMotorPidConfig retrieves PID configuration for a specific motor.
```

11.0.9 func (s *ConfigurationService) ResetToDefaults(ctx context.Context, req *starv1.ResetToDefaultsRequest) (*starv1.ResetToDefaultsResponse, error)

```
ResetToDefaults resets configuration to factory defaults. If persist_to_nvs
is true, saves defaults to NVS.
```

11.0.10 func (s *ConfigurationService) SaveConfiguration(ctx context.Context, req *starv1.SaveConfigurationRequest) (*starv1.SaveConfigurationResponse, error)

```
SaveConfiguration persists current in-memory configuration to NVS.
```

11.0.11 func (s *ConfigurationService) SetConfiguration(ctx context.Context, req *starv1.SetConfigurationRequest) (*starv1.SetConfigurationResponse, error)

```
SetConfiguration validates and applies new system configuration. Validates
all parameters before applying. If persist_to_nvs is true, saves to NVS.
```

11.0.12 func (s *ConfigurationService) SetMotorPidConfig(ctx context.Context, req *starv1.SetMotorPidConfigRequest) (*starv1.SetMotorPidConfigResponse, error)

```
SetMotorPidConfig updates PID configuration for a specific motor. Validates
PID parameters before applying. If persist_to_nvs is true, saves to NVS.
```

11.0.13 func (s *ConfigurationService) ValidateConfiguration(_ context.Context, req *starv1.ValidateConfigurationRequest) (*starv1.ValidateConfigurationResponse, error)

```
ValidateConfiguration validates configuration parameters without applying
them. Returns detailed validation results including any errors found.
```

11.0.14 type FirmwareService struct {

starv1.UnimplementedFirmwareUpdateServiceServer }

```
FirmwareService implements the FirmwareUpdateServiceServer interface.
This service handles over-the-air (OTA) firmware updates with validation and
rollback.
```

11.0.15 func NewFirmwareService() *FirmwareService

```
NewFirmwareService creates a new FirmwareService.
```

11.0.16 type GatewayService struct {

starv1.UnimplementedGatewayServiceServer

}

```
GatewayService implements the gRPC GatewayService for ROS2 <-> UI bridging.

Architecture:

    ROS2 (C++) <-> gRPC <-> GatewayService (Go) <-> WebSocket <-> UI (TypeScript)

Data flows:
 1. Telemetry (ROS2 -> UI): ROS2 calls ForwardTelemetry() -> cached ->
    WebSocket streams to UI
 2. Teleop (UI -> ROS2): UI sends via WebSocket -> UpdateTeleopCommand() ->
    ROS2 polls GetTeleopCommand()
```

```
3. PID Gains (UI -> ROS2): UI sends via WebSocket -> SetPIDGains() -> ROS2
   -> SPI -> RX72N
```

11.0.17 func NewGatewayService() *GatewayService

```
NewGatewayService creates a new GatewayService instance.
```

11.0.18 func (s *GatewayService) ForwardTelemetry(

ctx context.Context, req *starv1.ForwardTelemetryRequest,) (*starv1.ForwardTelemetryResponse, error)

```
ForwardTelemetry receives telemetry data from ROS2 and caches it for UI
streaming.

Called by ROS2 bridge node at 10 Hz. This is a non-blocking operation that
simply caches the telemetry data for WebSocket handlers to retrieve.
```

11.0.19 func (s *GatewayService) GetActiveClientCount() int32

```
GetActiveClientCount returns the number of connected WebSocket clients.
```

11.0.20 func (s *GatewayService) GetLatestTelemetry() (

*starv1.SystemStatus, *starv1.TelemetryData,)

```
GetLatestTelemetry returns the cached telemetry data for UI streaming.

Called by WebSocket handler to stream telemetry to UI clients. Returns nil
if no telemetry has been received yet.
```

11.0.21 func (s *GatewayService) GetTelemetryAge() time.Duration

```
GetTelemetryAge returns how old the cached telemetry is.

Useful for UI to display data freshness indicator.
```

11.0.22 func (s *GatewayService) GetTeleopCommand(

ctx context.Context, req *starv1.GetTeleopCommandRequest,) (*starv1.GetTeleopCommandResponse, error)

```
GetTeleopCommand returns the latest teleop command for ROS2 to execute.

Called by ROS2 bridge node at 50 Hz. Returns cached command with age in ms.
ROS2 is responsible for checking staleness (rejects commands > 500ms old).
```

11.0.23 func (s *GatewayService) Hub() HubNotifier

```
Hub returns the current HubNotifier under the read lock. Intended for use in
tests that need to inspect the wired hub without accessing the unexported
field directly, which would race with concurrent SetHub or ForwardTelemetry
calls.
```

11.0.24 func (s *GatewayService) RegisterClient(clientID string)

```
RegisterClient increments the active WebSocket client count.

Called when a new WebSocket client connects.
```

11.0.25 func (s *GatewayService) SetHub(h HubNotifier)

```
SetHub wires the WebSocket hub into the service so ForwardTelemetry can push
STAREnvelope messages directly to connected UI clients.

SetHub is safe to call concurrently with ForwardTelemetry and other hub
```

```
readers because it synchronises via hubMu (Lock/RLock). It is intended to
wire a HubNotifier and may also be used to replace s.hub at runtime.
```

11.0.26 func (s *GatewayService) SetPIDGains(

ctx context.Context, req *starv1.SetPIDGainsRequest,) (*starv1.SetPIDGainsResponse, error)

```
SetPIDGains updates PID gains for motor velocity control.

Called by Gateway when UI requests PID tuning. This is a placeholder
implementation that returns success but doesn't actually forward to ROS2 yet
(will be implemented when ROS2 PID service is available).
```

11.0.27 func (s *GatewayService) UnregisterClient(clientID string)

```
UnregisterClient decrements the active WebSocket client count.

Called when a WebSocket client disconnects.
```

11.0.28 func (s *GatewayService) UpdateTeleopCommand(cmd *starv1.VelocityCommand)

```
UpdateTeleopCommand updates the cached teleop command from UI.

Called by WebSocket handler when UI sends a new controller input. This
command will be polled by ROS2 via GetTeleopCommand().
```

11.0.29 type HubNotifier interface {

Broadcast(ctx context.Context, env *starv1.STAREnvelope) error }

```
HubNotifier is the interface GatewayService uses to push STAREnvelope
messages to the WebSocket hub. Defined here (not in ws) to avoid circular
imports; ws.Hub satisfies it.
```

11.0.30 type MotorControlService struct {

starv1.UnimplementedMotorControlServiceServer

}

```
MotorControlService implements the MotorControlServiceServer interface.
This service handles low-latency differential drive control.

Architecture: - Uses WireMessage wrapper for protocol multiplexing
(solves ambiguity) - Uses Dispatcher for centralized message routing
(solves concurrency race) - Uses structured logging (slog) for production
observability
```

11.0.31 func NewMotorControlService(h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *MotorControlService

```
NewMotorControlService creates a new MotorControlService. Requires HARQ
handler, Dispatcher for receiving telemetry, and logger.
```

11.0.32 func (s *MotorControlService) ControlStream(stream starv1.MotorControlService_ControlStreamServer) error

```
ControlStream handles bidirectional streaming. Client streams velocity
commands in, server streams encoder feedback out. Uses Dispatcher to receive
telemetry and wraps commands in WireMessage.
```

11.0.33 func (s *MotorControlService) EmergencyStop(ctx context.Context, req *starv1.EmergencyStopRequest) (*starv1.EmergencyStopResponse, error)

```
EmergencyStop triggers an immediate stop using dedicated
EmergencyStopCommand. This enables the RX72N to enter MOTOR_STATE_ESTOP and
engage hardware safety features.

TODO (tracked in GitHub issue): Implement priority framing for E-Stop. The
frame layer supports FlagPriority, but the HARQ interface doesn't expose it.
EmergencyStop should use priority framing to ensure immediate processing by
RX72N.
```

11.0.34 func (s *MotorControlService) SetMotorPower(ctx context.Context, req *starv1.SetMotorPowerRequest) (*starv1.SetMotorPowerResponse, error)

```
SetMotorPower sets raw motor power (bypass PID).
```

11.0.35 func (s *MotorControlService) SetVelocity(ctx context.Context, req *starv1.SetVelocityRequest) (*starv1.SetVelocityResponse, error)

```
SetVelocity sets wheel velocities for differential drive. Wraps the command
in WireMessage for protocol multiplexing and sends via HARQ.
```

11.0.36 func (s *MotorControlService) StreamEncoders(req *starv1.StreamEncodersRequest, stream starv1.MotorControlService_StreamEncodersServer) error

```
StreamEncoders streams encoder readings from the motor controller. Uses
Dispatcher to receive TelemetryData messages without contention.
```

11.0.37 type TelemetryService struct {

starv1.UnimplementedTelemetryServiceServer

}

```
TelemetryService implements the TelemetryServiceServer interface.
This service handles real-time sensor data aggregation and system health
monitoring.

Architecture: - Uses WireMessage wrapper for protocol multiplexing - Uses
Dispatcher for centralized message routing - Uses telemetryHolder for
thread-safe caching of latest telemetry - Background goroutine updates
cached telemetry from dispatcher
```

11.0.38 func NewTelemetryService(ctx context.Context, h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *TelemetryService

```
NewTelemetryService creates a new TelemetryService. Requires a context for
lifecycle management, HARQ handler for sending requests, Dispatcher for
receiving telemetry, and logger.
```

11.0.39 func (s *TelemetryService) GetSystemStatus(_ context.Context, req *starv1.GetSystemStatusRequest) (*starv1.GetSystemStatusResponse, error)

```
GetSystemStatus requests system status from the RX72N firmware. TODO:
Implement proper SystemStatusRequest message in wire.proto and send via
HARQ. For now, returns mock status until firmware integration is complete.
```

11.0.40 func (s *TelemetryService) GetTelemetry(_ context.Context, req *starv1.GetTelemetryRequest) (*starv1.GetTelemetryResponse, error)

```
GetTelemetry returns a snapshot of the latest telemetry data. Uses cached
telemetry updated by background goroutine for low latency.
```

11.0.41 func (s *TelemetryService) Shutdown()

Shutdown gracefully stops the TelemetryService background goroutines.

11.0.42 func (s *TelemetryService) StreamTelemetry(req *starv1.StreamTelemetryRequest, stream starv1.TelemetryService_StreamTelemetryServer) error

StreamTelemetry streams telemetry data at the requested rate (1-100 Hz).
Uses Dispatcher to receive telemetry and sends at controlled intervals.

12 Package: internal.testutil

Package testutil provides testing utilities and mocks for star-gateway.

STAR Project - Texas A&M University January 2026

12.0.1 CONSTANTS

12.0.2 const (

DefaultHARQTimeout = 50 * time.Millisecond)

12.0.3 FUNCTIONS

12.0.4 func NewDiscardLogger() *slog.Logger

```
NewDiscardLogger creates a logger that discards all output (for testing).
```

12.0.5 TYPES

12.0.6 type MockDispatcher struct {

SubscribeChan chan *dispatcher.DispatchedMessage SubscribeFunc func(msgType dispatcher.MessageType) <-chan *dispatcher.DispatchedMessage UnsubscribeFunc func(msgType dispatcher.MessageType, ch <-chan *dispatcher.DispatchedMessage) StartFunc func() error StopFunc func() error GetStateFunc func() dispatcher.State }

```
MockDispatcher is a mock implementation of the dispatcher.Dispatcher
interface.
```

12.0.7 func (m *MockDispatcher) GetState() dispatcher.State

12.0.8 func (m *MockDispatcher) Start(ctx context.Context) error

12.0.9 func (m *MockDispatcher) Stop() error

12.0.10 func (m *MockDispatcher) Subscribe(msgType dispatcher.MessageType) <-chan *dispatcher.DispatchedMessage

12.0.11 func (m *MockDispatcher) Unsubscribe(msgType dispatcher.MessageType, ch <-chan *dispatcher.DispatchedMessage)

12.0.12 type MockHARQ struct {

SendFunc func(ctx context.Context, data []byte, p ...harq.Priority) error SendWithTypeFunc func(ctx context.Context, data []byte, frameType frame.Type) error ReceiveFunc func(ctx context.Context) (*harq.ReceiveResult, error) GetStateFunc func() harq.State GetTxSeqFunc func() uint16 GetRxSeqFunc func() uint16 ResetFunc func() LastSentPayload []byte

ReceiveChan chan *harq.ReceiveResult }

```
MockHARQ is a mock implementation of the harq.HARQ interface.
Also implements the frameTypeSender interface (SendWithType) used by
performResetHandshake.
```

12.0.13 func (m *MockHARQ) GetLastSentPayload() []byte

```
GetLastSentPayload returns a copy of the last sent payload in a thread-safe
manner.
```

12.0.14 func (m *MockHARQ) GetRxSequence() uint16

12.0.15 func (m *MockHARQ) GetState() harq.State

12.0.16 func (m *MockHARQ) GetTxSequence() uint16

12.0.17 func (m *MockHARQ) Receive(ctx context.Context) (*harq.ReceiveResult, error)

12.0.18 func (m *MockHARQ) Reset()

12.0.19 func (m *MockHARQ) Send(ctx context.Context, data []byte, p ...harq.Priority) error

12.0.20 func (m *MockHARQ) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

```
SendWithType sends data with an explicit frame type. Used by
performResetHandshake to send FrameTypeReset frames.
```

13 Package: internal.transport

Package transport provides the USB CDC transport layer for RPi5 <-> RX72N communication.

The Raspberry Pi 5 communicates with the RX72N over USB CDC (Communications Device Class), which appears as a virtual serial port (e.g., /dev/ttyACM0). This provides a simpler interface than SPI and includes built-in flow control and reliability. This file helps us manage the raw serial communication over CDC. Doesnt check for any errors.

Reference: docs/sections/01_nanopb_protocol.tex

STAR Project - Texas A&M University January 2026

Package transport provides the low-level communication interfaces for the Gateway. This includes both real hardware (SPI) and simulation (Unix socket).

STAR Project - Texas A&M University January 2026

Package transport provides socket-based transport for HIL simulation.

Package transport provides the SPI transport layer for RPi5 <-> RX72N communication.

The Raspberry Pi 5 acts as the SPI controller, communicating with the RX72N peripheral at 10 MHz using SPI Mode 0.

Reference: docs/sections/01_nanopb_protocol.tex

STAR Project - Texas A&M University December 2025

13.0.1 CONSTANTS

13.0.2 const (

DefaultCDCDevice = "/dev/ttyACM0"

DefaultBaudRate = 115200

DefaultCDCTimeout = 100 * time.Millisecond

CDCDataBits = 8

RenesasVID = 0x045B

RX72NPID = 0x0235)

```
CDC configuration constants.
```

13.0.3 const (

DefaultSocketPath = "/tmp/star_rx72n.sock"

DefaultSocketTimeout = 100 * time.Millisecond

TransferSize = frame.MaxFrameSize)

```
Socket transport configuration constants.
```

13.0.4 const (

DefaultDevice = "/dev/spidev0.0"

DefaultSpeedHz = 10_000_000

DefaultMode = 0

DefaultBitsPerWord = 8

DefaultTimeout = 100 * time.Millisecond)

```
SPI configuration constants from hardware specification.
```

13.0.5 VARIABLES

13.0.6 var (

ErrDeviceNotFound = errors.New(“cdc: device not found”)

ErrReadTimeout = errors.New(“cdc: read timeout”))

```
Predefined CDC-specific errors.
```

13.0.7 var (

ErrNotImplemented = errors.New(“transport: not implemented”)

ErrDeviceNotOpen = errors.New(“transport: device not open”)

ErrTimeout = errors.New(“transport: operation timed out”)

ErrTransferFailed = errors.New(“transport: transfer failed”))

```
Predefined errors for transport operations.
```

13.0.8 TYPES

13.0.9 type CDCConfig struct {

Device string

BaudRate int

Timeout time.Duration

VID uint16

PID uint16 }

```
CDCConfig holds USB CDC configuration parameters.
```

13.0.10 func DefaultCDCConfig() *CDCConfig

```
DefaultCDCConfig returns the default CDC configuration.
```

13.0.11 type CDCTransport struct {

}

```
CDCTransport implements the Device interface for USB CDC communication. It
provides context-aware operations for serial communication with the RX72N.

Unlike SPI (which is full-duplex), CDC is half-duplex:
  - Send() writes data
  - Receive() reads data
  - Transfer() writes then reads (simulated full-duplex)
```

13.0.12 func NewCDCTransport(config *CDCConfig) *CDCTransport

```
NewCDCTransport creates a new CDCTransport with the given configuration.
If config is nil, uses DefaultCDCConfig().
```

13.0.13 func (c *CDCTransport) Close() error

```
Close releases the CDC device.
```

13.0.14 func (c *CDCTransport) Config() *CDCConfig

```
Config returns a copy of the current CDC configuration. Returns a copy to
prevent external mutation of transport settings.
```

13.0.15 func (c *CDCTransport) IsOpen() bool

```
IsOpen returns whether the device is currently open.
```

13.0.16 func (c *CDCTransport) Open() error

```
Open initializes the CDC device. If Device is empty in config, attempts to
auto-detect using VID/PID.
```

13.0.17 func (c *CDCTransport) Receive(maxLen int) ([]byte, error)

```
Receive reads data from CDC. Reads up to maxLen bytes from the serial port.
Returns ErrReadTimeout if no data is available.
```

13.0.18 func (c *CDCTransport) Send(data []byte) (int, error)

```
Send transmits data over CDC. Ensures the full buffer is sent by looping
until all bytes are written. Returns the number of bytes sent and any error.
```

13.0.19 func (c *CDCTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

```
Transfer performs a half-duplex transfer with context support. For CDC
(half-duplex):
 1. Sends txData
 2. Waits for response
 3. Reads response (same length as txData)

The context deadline is checked before and during the transfer. Uses a write
lock because SetReadTimeout mutates port state.
```

13.0.20 type Device interface {

Transfer(ctx context.Context, txData []byte) ([]byte, error)

Open() error

Close() error

Receive(len int) ([]byte, error)

Send(data []byte) (int, error)

IsOpen() bool }

```
Device represents the low-level connection (SPI or Mock/Socket).
This interface enables dependency injection for testing and simulation,
allowing the Gateway to communicate with either real hardware or a Virtual
RX72N simulator without code changes.

Device is an alias for Transport with the addition of context-aware
operations. Both SPITransport and SocketTransport implement this interface.
```

13.0.21 type SPIConfig struct {

Device string

SpeedHz uint32

Mode uint8

BitsPerWord uint8

Timeout time.Duration }

SPIConfig holds SPI configuration parameters.

13.0.22 func DefaultConfig() *SPIConfig

DefaultConfig returns the default SPI configuration.

13.0.23 type SPITransport struct {

}

SPITransport implements the Device interface using periph.io for SPI communication. The Raspberry Pi 5 acts as the SPI controller, communicating with the RX72N at 10 MHz.

13.0.24 func NewSPITransport(config *SPIConfig) *SPITransport

NewSPITransport creates a new SPITransport with the given configuration. If config is nil, uses DefaultConfig().

13.0.25 func (s *SPITransport) Close() error

Close releases the SPI device. Closes the periph.io SPI port and marks the transport as closed.

13.0.26 func (s *SPITransport) Config() *SPIConfig

Config returns the current SPI configuration.

13.0.27 func (s *SPITransport) IsOpen() bool

IsOpen returns whether the device is currently open.

13.0.28 func (s *SPITransport) Open() error

Open initializes the SPI device. Initializes periph.io host drivers, opens the SPI port, and configures it with the specified mode, speed, and bits per word.

13.0.29 func (s *SPITransport) Receive(maxLen int) ([]byte, error)

Receive reads data from SPI. Performs a read-only operation by sending dummy bytes (zeros). SPI is inherently full-duplex, so we must write while reading.

13.0.30 func (s *SPITransport) Send(data []byte) (int, error)

Send transmits data over SPI. Performs a write-only operation by discarding the received data. SPI is inherently full-duplex, so we must read while writing.

13.0.31 func (s *SPITransport) SetReadDeadline(deadline time.Time) error

SetReadDeadline sets a deadline for future Receive calls. Note: SPI transactions are synchronous and cannot be interrupted once started. The deadline is checked before each Receive.

13.0.32 func (s *SPITransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

Transfer performs full-duplex SPI transfer with context support. Sends txData while simultaneously receiving data of the same length. This is the native SPI operation mode.

The context deadline is checked before the transfer. Note that SPI transactions are synchronous and cannot be interrupted once started.

13.0.33 type SocketTransport struct {

}

```
SocketTransport implements the Device interface using Unix Domain Sockets.
It uses the "Unlocking" pattern to ensure thread-safe, non-blocking
shutdowns.
```

13.0.34 func NewSocketTransport(socketPath string) *SocketTransport

```
NewSocketTransport creates a new SocketTransport with the given socket path.
```

13.0.35 func (s *SocketTransport) Close() error

```
Close closes the socket connection.

Uses the "Unlocking" pattern: sets s.conn to nil under lock, then calls
conn.Close() outside the lock. This ensures pending Transfer() calls are
interrupted immediately (returning net.ErrClosed) without waiting for a
mutex.
```

13.0.36 func (s *SocketTransport) IsOpen() bool

```
IsOpen returns whether the socket is currently connected.
```

13.0.37 func (s *SocketTransport) Open() error

```
Open establishes a connection to the Virtual RX72N process via Unix socket.
```

13.0.38 func (s *SocketTransport) Path() string

```
Path returns the Unix socket path.
```

13.0.39 func (s *SocketTransport) Receive(maxLen int) ([]byte, error)

```
Receive implements the legacy Transport interface.
```

13.0.40 func (s *SocketTransport) Send(data []byte) (int, error)

```
Send implements the legacy Transport interface.
```

13.0.41 func (s *SocketTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

```
Transfer sends txData and receives the response.

Uses the "Unlocking" pattern: the mutex is held ONLY to retrieve the
connection reference. The actual I/O is performed without the lock, allowing
Close() to interrupt this operation immediately.
```

14 Package: internal.ws

Package ws provides WebSocket support for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides the WebSocket transport layer for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides WebSocket support for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides the WebSocket transport layer for the STAR Gateway's L5 (application/session) service layer.

Architecture role: ws sits between the HTTP server (net/http) and the GatewayService (internal/service). It manages the lifecycle of individual WebSocket connections (Client), co-ordinates fan-out to all connected UI clients (Hub), and serialises/deserialises binary Protobuf STAREnvelope frames. GatewayService communicates with the hub exclusively through the HubNotifier interface so ws can be replaced or mocked in tests without touching service logic.

Thread-safety contract for implementations and consumers:

- Hub.Broadcast and Hub.Run are safe to call concurrently from different

  ```
  goroutines; Broadcast enqueues via a channel and never accesses the clients
  map directly.
  ```

- Hub.register and Hub.unregister channels are the ONLY external entry-points

  ```
  for modifying the clients map; all mutations happen inside Run's goroutine.
  ```

- Client.writePump is the ONLY goroutine that writes to the WebSocket

  ```
  connection; external code must not call conn.WriteMessage directly.
  ```

- MotorController.EmergencyStop may be called from Client.readPump while the

  ```
  connection is alive; implementations must be safe for concurrent calls.
  ```

STAR Project - Texas A&M University February 2026

14.0.1 CONSTANTS

14.0.2 const (

EstopTimeout = 5 * time.Second)

14.0.3 VARIABLES

14.0.4 var ErrBroadcastFull = errors.New("ws: hub broadcast channel full")

```
ErrBroadcastFull is returned by Broadcast when the hub's outbound channel is
saturated and the envelope must be dropped to avoid blocking.
```

14.0.5 var ErrInvalidEnvelope = errors.New("ws: nil envelope passed to Broadcast")

```
ErrInvalidEnvelope is returned by Broadcast when the caller passes a nil
envelope.
```

14.0.6 FUNCTIONS

14.0.7 func NewHandler(hub *Hub, motorService MotorController, logger *slog.Logger, allowInsecureWebsocket bool) http.Handler

```
NewHandler returns an http.Handler that upgrades each request to a WebSocket
connection and wires it into hub.

allowInsecureWebsocket controls origin checking for incoming upgrade
requests: when true, all origins are permitted -- useful for local-network
or development deployments where requests arrive from a different port or
host. When false, a same-origin policy is enforced -- the Origin header must
match the request Host. Pass false for production deployments; pass true
only for local/dev environments.

Panics at construction time if hub or motorService is nil:
  - A nil hub has no client set and no event loop, so registration would
    block indefinitely and routing would be unreachable.
  - A nil motorService would cause a nil-pointer dereference in
    Client.readPump on e-stop frames.
```

14.0.8 TYPES

14.0.9 type Client struct {

}

```
Client represents a single connected WebSocket client managed by the Hub.
Each Client owns two goroutines -- readPump (reads frames from the
connection) and writePump (serialises frames onto the connection) -- and
communicates with the Hub via the send channel and the hub.register /
hub.unregister channels. motorService is called synchronously in readPump
for e-stop frames. logger is used for structured diagnostic output; it is
never nil.
```

14.0.10 func NewClient(hub *Hub, conn *websocket.Conn, motorService MotorController, logger *slog.Logger) *Client

```
NewClient creates a properly initialised Client and registers it with
the provided hub. It is the canonical constructor for external callers;
the hub's handler (ws.NewHandler) uses this to ensure all unexported fields
are set.

Callers must start the read and write pump goroutines after construction:

    go client.writePump()
    client.readPump()
```

14.0.11 type Hub struct {

}

```
Hub maintains the set of active clients and broadcasts messages to the
clients.
```

14.0.12 func NewHub(logger *slog.Logger) *Hub

```
NewHub creates a new Hub with the provided logger.
```

14.0.13 func (h *Hub) Broadcast(ctx context.Context, env *starv1.STAREnvelope) error

```
Broadcast pushes env to all connected clients. Non-blocking: if the hub's
internal channel is full, the envelope is dropped and ErrBroadcastFull
is returned so callers can decide whether to log or meter the drop.
If ctx is already cancelled when Broadcast is called, ctx.Err() is returned
immediately.

Ownership transfer: the caller must not access or reuse env after calling
Broadcast. The Hub's event loop (stampAndFanOut) mutates env.Seq and
env.TimestampUs in-place before marshalling and fan-out.
```

14.0.14 func (h *Hub) ClientCount() int

```
ClientCount returns the current number of active registered WebSocket
clients. The value is read atomically and is safe to call from any goroutine
without locking. Intended for use by metrics collection and health checks.
```

14.0.15 func (h *Hub) Run(ctx context.Context)

```
Run starts the hub's main event loop. It blocks until ctx is cancelled and
must therefore be invoked in its own goroutine so callers do not block.

Concurrency guarantees:
  - Client registration and unregistration are safe from any goroutine via
    the h.register / h.unregister channels; internal client state is never
    accessed directly from outside Run.
  - Outbound broadcasts are delivered from the h.broadcast channel;
    use Broadcast to enqueue messages without racing with the event loop.
  - When ctx is cancelled, Run closes every active client's send channel
    before returning so writePump goroutines terminate cleanly.
```

14.0.16 func (h *Hub) SetInboundDispatcher(d InboundDispatcher)

```
SetInboundDispatcher registers an InboundDispatcher that receives all
inbound WebSocket envelopes (non-e-stop payloads forwarded by client
readPumps). Must be called before Hub.Run starts. Panics if called after
Run has started to enforce the "set-before-start" invariant and prevent data
races.
```

14.0.17 type HubNotifier interface {

Broadcast(ctx context.Context, env *starv1.STAREnvelope) error }

```
HubNotifier is the minimal interface GatewayService uses to push messages to
the WebSocket hub.
```

14.0.18 type InboundDispatcher interface {

Dispatch(ctx context.Context, env *starv1.STAREnvelope) error }

```
InboundDispatcher receives envelopes forwarded inbound by WebSocket clients
(all non-e-stop payloads read from the browser/UI). Implement this interface
and register it with Hub.SetInboundDispatcher to route commands to the
application layer instead of silently dropping them.
```

14.0.19 type MotorController interface {

EmergencyStop(ctx context.Context, reason string) error }

```
MotorController is the interface for motor safety operations.
Implementations receive a context for cancellation and tracing; they may
ignore cancellation but should propagate it to any downstream call that
supports context.
```